



# Proceso de Desarrollo de Software

## Parte 2:

## Patrones de Diseño



**Idea principal:** los patrones son soluciones conocidas a problemas conocidos; ayudan a reutilizar experiencia, comunicar diseño y reducir improvisación.

**Clasificación útil:** creacionales (creación de objetos), estructurales (composición/adaptación) y de comportamiento (cómo colaboran los objetos en ejecución).

**MVC:** aparece como patrón arquitectónico para separar modelo, vista y controlador.

**Cómo estudiar:** para cada patrón conviene recordar: problema que resuelve, componentes, cuándo elegirlo, ejemplo típico y advertencias.

**Pregunta clásica de parcial:** dado un escenario, reconocer la “señal” del problema y justificar por qué un patrón encaja mejor que otro.

Tip: si en el enunciado aparecen muchos if/switch repetidos, acoplamiento fuerte o cambios frecuentes, probablemente hay un patrón útil detrás.

| Patrón           | Tipo           | Lo elijo cuando...                                                                | Señal de examen / tip rápido                                                            |
|------------------|----------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>MVC</b>       | Arquitectónico | querés separar UI, lógica de negocio y coordinación de eventos                    | pensá en apps con interfaz y responsabilidades claras entre modelo, vista y controlador |
| <b>Strategy</b>  | Comportamiento | hay varias formas de hacer la misma tarea y querés intercambiarlas en runtime     | si cambia el algoritmo pero no la intención, Strategy suele encajar                     |
| <b>Adapter</b>   | Estructural    | necesitás usar una clase/API existente con interfaz incompatible                  | si ya existe algo útil pero “no habla tu idioma”, adaptalo                              |
| <b>Composite</b> | Estructural    | tenés jerarquías árbol y querés tratar hojas y grupos de forma uniforme           | objetos simples + compuestos con misma interfaz = Composite                             |
| <b>Decorator</b> | Estructural    | querés agregar responsabilidades sin tocar la clase base ni multiplicar subclases | si la herencia explota en combinaciones, pensá Decorator                                |

| Patrón                | Tipo           | Lo elijo cuando...                                                               | Señal de examen / tip rápido                                              |
|-----------------------|----------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>State</b>          | Comportamiento | el comportamiento cambia según el estado interno del objeto                      | si hay muchos estados y condicionales por estado, pensá State             |
| <b>Observer</b>       | Comportamiento | hay eventos y varios interesados deben reaccionar sin acoplarse al emisor        | uno publica y muchos reaccionan: Observer                                 |
| <b>Singleton</b>      | Creacional     | debe existir una única instancia accesible globalmente                           | útil para un recurso único; peligroso si se vuelve “global” sin necesidad |
| <b>Simple Factory</b> | Creacional     | necesitás centralizar creación según un parámetro y la creación es algo compleja | una fábrica central con if/switch y configuración                         |
| <b>Factory Method</b> | Creacional     | querés delegar creación a subclases y extender sin modificar la fábrica central  | si querés crecer por herencia/polimorfismo, Factory Method                |

## ARQUITECTÓNICO

**¿Qué hace?**

Separa la aplicación en Modelo, Vista y Controlador.  
Favorece la separación de intereses entre datos/reglas, presentación y manejo de eventos.

**Ejemplo breve**

Aplicación web de productos: el Modelo valida reglas, la Vista muestra formularios/listados y el Controlador procesa acciones del usuario.

**¿Cuándo elegirlo?**

Cuando la aplicación tiene interfaz y conviene aislar lógica de negocio de la UI.  
Cuando querés mejorar reusabilidad y ordenar responsabilidades.

**Consideraciones**

Ventajas: separación, flexibilidad y reuso.  
Desventajas: puede agregar complejidad y no siempre conviene en vistas muy simples.  
Conviene validar datos en varios niveles: vista, controlador y modelo.

## COMPORTAMIENTO

### ¿Qué hace?

Encapsula distintos algoritmos para resolver el mismo problema.

Permite cambiar la forma de resolver una tarea en tiempo de ejecución.

### Ejemplo breve

Cálculo de costo de envío: estrategia Normal, Express o Retiro en sucursal; el contexto delega en la estrategia elegida.

### ¿Cuándo elegirlo?

Cuando un objeto debe hacer “lo mismo” pero con algoritmos distintos.

Cuando querés evitar condicionales grandes para elegir entre variantes de cálculo/comportamiento.

### Consideraciones

Buena opción para Open/Closed: agregás nuevas estrategias sin tocar el contexto.

Pedí que todas las estrategias compartan una interfaz clara.

No conviene si solo existe una variante estable y no habrá cambios.

**Pista para el parcial:** si lo que cambia es el algoritmo, pero no el objeto ni su estado interno, Strategy suele ser mejor que State.

## ESTRUCTURAL

### ¿Qué hace?

Envuelve una clase existente con una interfaz compatible con lo que el cliente necesita.

Permite integrar componentes externos o legados sin modificar su código.

### Ejemplo breve

Módulo de notificaciones que espera enviar(mensaje), pero el proveedor externo ofrece sendSMS(text, number): el adapter traduce ambas interfaces.

### ¿Cuándo elegirlo?

Cuando ya existe una clase/API útil pero su interfaz no coincide con la requerida.

Cuando querés seguir diseñando contra una abstracción aunque la implementación externa todavía no esté definida.

### Consideraciones

Reduce acoplamiento con proveedores externos.

Conviene que el cliente dependa de una interfaz propia del dominio.

No soluciona mala lógica de negocio; solo adapta interfaces.

**Pista para el parcial:** si el problema principal es “forma de hablar distinta”, pensá Adapter.

## ESTRUCTURAL

### ¿Qué hace?

Compone objetos en una estructura de árbol.  
Permite tratar de forma uniforme objetos individuales y conjuntos compuestos.

### Ejemplo breve

Editor gráfico: Círculo, Cuadrado y Triángulo son hojas;  
Grupo es un compuesto que puede contener figuras y otros grupos.

### ¿Cuándo elegirlo?

Cuando hay jerarquías parte-todo (archivos/carpetas, figuras/grupos, menús/submenús).  
Cuando el cliente debería usar hojas y compuestos con la misma interfaz.

### Consideraciones

Muy útil para operaciones recursivas.  
Diseña bien la interfaz componente para no forzar a las hojas a implementar operaciones sin sentido.  
Ideal cuando el dominio naturalmente forma árboles.

**Pista para el parcial:** si el enunciado habla de grupos que contienen elementos y subgrupos, Composite es candidato fuerte.

## ESTRUCTURAL

### ¿Qué hace?

Agrega responsabilidades a objetos concretos de manera dinámica y transparente.

Envuelve un componente para extender comportamiento sin modificar su clase.

### Ejemplo breve

Sistema de café: bebida base + leche + azúcar + chocolate; cada agregado decora la bebida y suma comportamiento/costo.

### ¿Cuándo elegirlo?

Cuando querés combinar funcionalidades opcionales sin explotar la jerarquía de herencia.

Cuando las responsabilidades pueden agregarse y quitarse.

### Consideraciones

Más flexible que la herencia estática.

Puede generar muchos objetos pequeños encadenados y complicar debugging.

El decorador y el componente decorado no son el mismo objeto.

**Pista para el parcial:** si aparecen “extras acumulables” o combinaciones de funcionalidades, Decorator es una muy buena señal.

## COMPORTAMIENTO

### ¿Qué hace?

Representa cada estado relevante como un objeto/clase.  
Hace que el comportamiento cambie cuando cambia el estado interno.

### Ejemplo breve

Cajero automático: SinTarjeta, EsperandoPIN, Autenticado y SinEfectivo; cada estado responde distinto a la misma operación.

### ¿Cuándo elegirlo?

Cuando el comportamiento depende del estado y este cambia en ejecución.  
Cuando un método está lleno de if/else o switch basados en estados.

### Consideraciones

Ayuda a eliminar condicionales largos.  
Conviene modelar bien transiciones válidas entre estados.  
Si hay pocos estados estables, quizá un enum simple alcance.

**Pista para el parcial:** si una misma acción responde distinto según “fase” o “estado”, compará State antes de quedarte con condicionales.

## COMPORTAMIENTO

### ¿Qué hace?

Permite notificar a interesados cuando ocurre un evento en un sujeto.

Desacopla al emisor del evento de quienes reaccionan a él.

### Ejemplo breve

Estación meteorológica: app de usuario, central de emergencias y servicio de estadísticas se suscriben a cambios en las mediciones.

### ¿Cuándo elegirlo?

Cuando varios objetos deben reaccionar a un cambio/evento.

Cuando no te interesa conocer de antemano cuántos observadores habrá ni quiénes serán.

### Consideraciones

Muy útil en sistemas orientados a eventos.

Evita observadores que generen cascadas difíciles de seguir.

**Pista para el parcial:** si uno emite eventos y muchos se suscriben/reaccionan, es un caso clásico de Observer.

## CREACIONAL

### ¿Qué hace?

Asegura que exista una única instancia de una clase.  
Ofrece un punto de acceso controlado a esa instancia.

### Ejemplo breve

ConfigurationManager: una sola instancia centraliza la configuración del sistema para todos los módulos.

### ¿Cuándo elegirlo?

Cuando realmente debe existir un único objeto compartido por todo el sistema.  
Cuando múltiples instancias traerían inconsistencia o desperdicio.

### Consideraciones

Usarlo con criterio: puede introducir acoplamiento global y dificultar tests.  
El constructor suele ocultarse para controlar la instancia.  
No lo elijas solo por comodidad.

**Pista para el parcial:** si el argumento principal es “necesitamos exactamente una instancia”, Singleton puede aplicar; si es solo acceso global, pensalo dos veces.

## CREACIONAL

### ¿Qué hace?

Centraliza en una clase la creación de objetos de una familia común.

Decide qué instancia devolver según un parámetro y/o lógica de configuración.

### Ejemplo breve

TransportFactory devuelve Truck o Bicycle según el tipo solicitado por el cliente.

### ¿Cuándo elegirlo?

Cuando en muchas partes del sistema se repite la misma lógica de instanciación.

Cuando crear los objetos implica cierta complejidad o configuración inicial.

### Consideraciones

Es un modismo muy útil, aunque no siempre se presenta como patrón GoF formal.

Suele usar if/else o switch.

Si empiezan a crecer mucho los casos o querés extender sin tocar la fábrica, considerá Factory Method.

**Pista para el parcial:** si querés un “creador centralizado” sencillo, empezá por Simple Factory.

## CREACIONAL

### ¿Qué hace?

Define un método de creación y delega a subclases la decisión de qué producto concreto instanciar.  
Encapsula creación usando herencia y polimorfismo.

### Ejemplo breve

Logistics define `createTransport()`; `RoadLogistics` crea `Truck` y `SeaLogistics` crea `Ship`.

### ¿Cuándo elegirlo?

Cuando querés extender los tipos creados sin modificar una fábrica central.  
Cuando la lógica de creación debe variar por subtipo de creador.

### Consideraciones

Mejora extensibilidad y acompaña `Open/Closed`.  
Puede agregar más clases que `Simple Factory`.  
Tiene sentido cuando el sistema probablemente crecerá con nuevas variantes.

**Pista para el parcial:** si el problema es crecimiento por nuevas variantes y no querés tocar código existente, `Factory Method` gana terreno frente a `Simple Factory`.

## ESTRUCTURAL

### ¿Qué hace?

Expone una interfaz simple para la utilización de un componente complejo de terceros. Permite que dos o más soluciones interactúen manteniendo el acoplamiento al mínimo posible.

### Ejemplo breve

Patio de comidas: en vez de coordinar cocinero, armador, cajero y despachador, el cliente hace un único `pedirCombo()`.

También sirve para simplificar un módulo exportador y dejar un único punto de entrada limpio para exportar.

### ¿Cuándo elegirlo?

Cuando necesitas ocultar la complejidad real de un subsistema detrás de una interfaz sencilla.

Cuando quieres usar un componente externo complejo sin acoplar fuertemente toda la solución a él.

### Consideraciones

La clase Facade reúne los componentes complejos o externos y expone métodos simples.

El cliente debería depender de la fachada y no del detalle interno del subsistema.

Si cambia el proceso interno, el cliente no tiene por qué cambiar.

**Pista para el parcial:** si el cliente tiene que conocer demasiados pasos, objetos o llamados para lograr una tarea, Facade es una señal muy fuerte.

## Adapter + Facade

### Enunciado funcional

Una app de e-commerce debe cotizar envíos con dos proveedores externos. Cada proveedor publica una API distinta y compleja. El resto del sistema solo debería poder invocar una operación simple: `cotizarEnvio(origen, destino, peso)` sin conocer ni los endpoints ni el formato propio de cada proveedor.

### Claves para identificar patrones

“APIs distintas” o “interfaces incompatibles” suele apuntar a Adapter.

“Una operación simple” para ocultar varios pasos o subsistemas sugiere Facade.

“No exponer detalles del proveedor al resto del sistema” refuerza bajo acoplamiento.

### Combinación sugerida

Adapter traduce cada API externa a una interfaz interna común.

Facade ofrece el punto de entrada simple que usa la aplicación.

## Strategy + Factory Method + Observer

### Enunciado funcional

Un sistema de pagos debe cobrar pedidos con distintos medios (tarjeta, transferencia y billetera virtual). El medio de pago se elige en tiempo de ejecución. Además, al confirmarse el cobro se debe notificar automáticamente al módulo de facturación, al de stock y al de analítica. El negocio espera poder agregar nuevos medios de pago sin reescribir la lógica existente.

### Claves para identificar patrones

“Distintos medios elegidos en tiempo de ejecución” es una pista clara de Strategy.

“Agregar nuevos medios sin tocar código existente” sugiere una creación extensible: Factory Method.

“Notificar automáticamente a varios módulos” es una señal clásica de Observer.

### Combinación sugerida

Strategy encapsula cómo se procesa cada forma de pago.

Factory Method ayuda a crear la estrategia concreta según el contexto sin if/switch dispersos.

Observer desacopla el evento de pago confirmado de los módulos que reaccionan.

## Composite + Decorator + State + Facade

### Enunciado funcional

Un editor de campañas de marketing permite armar piezas compuestas por bloques simples (texto, imagen, botón) y por secciones que contienen otros bloques. Cada pieza puede publicarse en distintos estados: borrador, aprobada o publicada, cambiando lo que se puede hacer en cada momento. Además, una campaña puede sumar opcionalmente marca de agua, compresión o tracking sin modificar el bloque original. Finalmente, el front-end solo debería llamar a `publicarCampaña()` sin conocer el proceso interno de validación, armado y envío.

### Claves para identificar patrones

“Bloques simples y secciones que contienen otros bloques” apunta a Composite.

“Agregar tracking, compresión o marca de agua opcionalmente” es señal de Decorator.

“Borrador / aprobada / publicada” con cambios de comportamiento sugiere State.

“Un único `publicarCampaña()` que oculta varios pasos” sugiere Facade.

### Combinación sugerida

Composite modela la estructura jerárquica de la campaña.

Decorator agrega responsabilidades opcionales a cada pieza.

State encapsula las restricciones según el estado de publicación.

Facade simplifica la interacción del cliente con todo el subsistema.

**State vs Strategy:** ambos encapsulan comportamiento, pero en State cambia por el estado interno del contexto; en Strategy se selecciona un algoritmo para una tarea.

**Simple Factory vs Factory Method:** ambos centralizan creación, pero Simple Factory concentra decisiones en una clase; Factory Method distribuye creación en subclases.

**Decorator vs Composite:** ambos “envuelven” o contienen objetos, pero Decorator agrega responsabilidades; Composite arma una estructura jerárquica parte-todo.

**Observer vs MVC:** Observer modela publicación/suscripción de eventos; MVC organiza responsabilidades de una aplicación con interfaz. Un sistema MVC puede usar Observer internamente.

**Adapter vs Strategy:** Adapter hace compatible una interfaz; Strategy cambia el algoritmo que se usa.

**Adapter vs Facade:** Adapter resuelve problemas de incompatibilidad entre interfaces existentes para que puedan trabajar juntas, mientras que la Facade simplifica el uso de un sistema complejo proporcionando una interfaz única y de alto nivel

Regla útil: antes de elegir un patrón, nombrá el problema con una frase simple. Si el problema está bien formulado, el patrón suele aparecer solo.

**Diseñar bien no es “usar patrones por moda”:** es reconocer problemas repetidos y elegir una solución clara, mantenible y justificable.

**En el ámbito profesional,** la calidad del diseño impacta en tiempo de desarrollo, facilidad de testeo, costo de cambios y comunicación del equipo.

**Los patrones ayudan a hablar un idioma común:** decir “esto parece Strategy” o “acá conviene un Adapter” acelera decisiones y evita discusiones ambiguas.

**Buena práctica clave:** priorizar bajo acoplamiento, alta cohesión, contratos claros e intención explícita del código.

**Para el parcial y para el trabajo real,** no alcanza con memorizar nombres: hay que justificar por qué un patrón resuelve mejor un contexto determinado.

Un buen diseño se nota cuando el sistema tolera cambios sin volverse frágil.

- ✓ Nombrar bien clases, interfaces y responsabilidades.
- ✓ Evitar duplicación de lógica y condicionales repetidos.
- ✓ Programar contra abstracciones cuando el problema lo amerita.
- ✓ Mantener ejemplos de uso y pruebas que documenten el comportamiento.
- ✓ Elegir el patrón más simple que resuelva el problema real; sobre-ingeniería también es un problema.
- ✓ Pensar siempre en extensibilidad, legibilidad y costo futuro de mantenimiento.

Aprender patrones no es coleccionar soluciones: es entrenar criterio de diseño.

- Espero que hayan disfrutado de la materia.
- Los conceptos que vimos son muy importantes, marcan la diferencia profesional para un programador.
- De mi parte quiero agradecerles por haber compartido juntos este cuatrimestre.
- Al igual que ustedes, ¡sigo aprendiendo! Disculpen los errores.
- Queda mucho más por ver, se que serán excelentes profesionales.

¡Muchas gracias!

Walter Matías Gómez